Here's a **complete explanation** of **Instance Variables and Methods** in Python OOP:

🎯 **WHAT ARE INSTANCE VARIABLES?**

Instance Variables are variables that:

→ Belong to a SPECIFIC OBJECT

→ Defined inside __init__ using self

→ Each object gets its OWN COPY

→ Different objects have DIFFERENT values

→ Created when object is instantiated

Simple Analogy:

Class → Human Blueprint

Object → Each individual person

Every person has:

→ Their OWN name (instance variable)

→ Their OWN age (instance variable)

→ Their OWN height (instance variable)

No two people SHARE these values!

🔬 **INSTANCE VARIABLE — DEEP DIVE**

python

```
class Student:

    def __init__(self, name, age, marks):
        # ↓ self = current object
        self.name  = name    # instance variable
        self.age   = age     # instance variable
        self.marks = marks   # instance variable
        self.grade = "A"     # instance variable
```

```
                    # with default value


# Creating 3 DIFFERENT objects
s1 = Student("Rahul", 20, 90)
s2 = Student("Priya", 22, 85)
s3 = Student("Amit",  21, 78)

# Each object has its OWN copy
print(s1.name,  s1.age, s1.marks)  # Rahul 20 90
print(s2.name,  s2.age, s2.marks)  # Priya 22 85
print(s3.name,  s3.age, s3.marks)  # Amit  21 78

# Changing one does NOT affect others
s1.marks = 95
print(s1.marks)   # 95  → changed
print(s2.marks)   # 85  → unchanged
print(s3.marks)   # 78  → unchanged
```

🧠 **HOW self WORKS**

python

```
class Dog:

    def __init__(self, name, breed):
        # self = the CURRENT object being created
        # self.name = attach 'name' to THIS object
        self.name  = name
        self.breed = breed



d1 = Dog("Tommy", "Labrador")
```

```
d2 = Dog("Bruno", "Poodle")

# When d1 is created:
# self = d1
# self.name  = d1.name  = "Tommy"
# self.breed = d1.breed = "Labrador"

# When d2 is created:
# self = d2
# self.name  = d2.name  = "Bruno"
# self.breed = d2.breed = "Poodle"

print(d1.name)   # Tommy
print(d2.name)   # Bruno
```

Visual of self:

```
Dog("Tommy", "Labrador")
      |
      ▼
  __init__(self, name, breed)
  self ———→ d1 object
  self.name  = "Tommy"    → stored in d1
  self.breed = "Labrador" → stored in d1

Dog("Bruno", "Poodle")
      |
      ▼
  __init__(self, name, breed)
  self ———→ d2 object
  self.name  = "Bruno"  → stored in d2
  self.breed = "Poodle" → stored in d2
```

📝 **TYPES OF INSTANCE VARIABLES**

Type 1 — Defined in __init__:

python

```
class Person:
    def __init__(self, name, age):
        self.name = name    # set from parameter
        self.age  = age     # set from parameter
        self.alive = True   # default value
        self.friends = []   # default empty list



p = Person("Rahul", 25)
print(p.name)     # Rahul
print(p.alive)    # True
print(p.friends)  # []
```

Type 2 — Defined in other methods:

python

```
class BankAccount:
    def __init__(self, holder):
        self.holder      = holder
        self.balance     = 0
        self.transaction = []    # empty list

    def deposit(self, amount):
        self.balance += amount
        self.last_deposit = amount   # new instance var
        self.transaction.append(
            f"Deposit: +{amount}"
        )

    def withdraw(self, amount):
```

```
        if amount <= self.balance:
            self.balance     -= amount
            self.last_withdraw = amount  # new instance var
            self.transaction.append(
                f"Withdraw: -{amount}"
            )
        else:
            print("Insufficient funds!")



acc = BankAccount("Rahul")
acc.deposit(10000)
acc.withdraw(3000)

print(acc.balance)       # 7000
print(acc.last_deposit)   # 10000
print(acc.last_withdraw)  # 3000
print(acc.transaction)
# ['Deposit: +10000', 'Withdraw: -3000']
```

Type 3 — Added dynamically:

python

```
class Car:
    def __init__(self, brand):
        self.brand = brand



car = Car("Toyota")
print(car.brand)    # Toyota

# Add new instance variable AFTER creation
car.color = "Red"
```

```
car.speed = 180
car.year  = 2024

print(car.color)    # Red
print(car.speed)    # 180
print(car.year)     # 2024

# Shows ALL instance variables
print(car.__dict__)
# {'brand': 'Toyota', 'color': 'Red',
#  'speed': 180, 'year': 2024}
```

🔧 **WHAT ARE INSTANCE METHODS?**

Instance Methods are functions that:

→ Belong to a SPECIFIC OBJECT

→ Always take 'self' as first parameter

→ Can ACCESS and MODIFY instance variables

→ Can access CLASS variables too

→ Called on an OBJECT (not class)

→ Most common type of method in Python

🔬 **INSTANCE METHOD — DEEP DIVE**

python

```
class Rectangle:

    def __init__(self, width, height):
        # Instance variables
        self.width  = width
        self.height = height

    # ── INSTANCE METHODS ──────────────────────
```

```
    def area(self):
        # Accessing instance variables via self
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def is_square(self):
        return self.width == self.height

    def scale(self, factor):
        # Modifying instance variables
        self.width  *= factor
        self.height *= factor

    def __str__(self):
        return f"Rectangle({self.width}x{self.height})"


# Creating objects
r1 = Rectangle(4, 6)
r2 = Rectangle(5, 5)

# Calling instance methods
print(r1.area())        # 24
print(r1.perimeter())    # 20
print(r1.is_square())    # False

print(r2.area())        # 25
print(r2.is_square())    # True
```

```
# Modifying via instance method
r1.scale(2)
print(r1)            # Rectangle(8x12)
print(r1.area())       # 96
```

🏦 **COMPLETE REAL WORLD EXAMPLE**

python

```
class BankAccount:

    # Class variable
    bank_name    = "SBI Bank"
    interest_rate = 0.05

    def __init__(self, holder, balance=0):
        # Instance variables
        self.holder      = holder
        self.balance     = balance
        self._pin        = 0000       # protected
        self.__logs      = []         # private

    # ── INSTANCE METHODS ──────────────────────

    def deposit(self, amount):
        """Add money to account"""
        if amount > 0:
            self.balance += amount
            self.__logs.append(
                f"Deposited: +₹{amount}"
            )
            return f"✅ Deposited ₹{amount}"
```

```
        return "❌ Invalid amount"

    def withdraw(self, amount):
        """Remove money from account"""
        if amount > self.balance:
            return "❌ Insufficient funds"
        if amount <= 0:
            return "❌ Invalid amount"
        self.balance -= amount
        self.__logs.append(
            f"Withdrawn: -₹{amount}"
        )
        return f"✅ Withdrawn ₹{amount}"

    def get_balance(self):
        """Check current balance"""
        return f"💰 Balance: ₹{self.balance}"

    def add_interest(self):
        """Add annual interest"""
        interest = self.balance * self.interest_rate
        self.balance += interest
        self.__logs.append(
            f"Interest Added: +₹{interest:.2f}"
        )
        return f"✅ Interest ₹{interest:.2f} added"

    def get_statement(self):
        """Get transaction history"""
        print(f"\n{'–'*30}")
        print(f"  {self.bank_name}")
```

```
        print(f"  Account: {self.holder}")
        print(f"{'–'*30}")
        for log in self.__logs:
            print(f"  {log}")
        print(f"{'–'*30}")
        print(f"  Final Balance: ₹{self.balance}")
        print(f"{'–'*30}\n")

    def transfer(self, amount, target_account):
        """Transfer to another account"""
        result = self.withdraw(amount)
        if " ✅ " in result:
            target_account.deposit(amount)
            return f" ✅ Transferred ₹{amount} to {target_account.holder}"
        return " ❌ Transfer failed"

    def __str__(self):
        return f"Account[{self.holder}] → ₹{self.balance}"

    def __repr__(self):
        return f"BankAccount('{self.holder}', {self.balance})"


# ── USING THE CLASS ──────────────────────────
acc1 = BankAccount("Rahul", 10000)
acc2 = BankAccount("Priya", 5000)

# Instance method calls
print(acc1.deposit(5000))     # ✅ Deposited ₹5000
print(acc1.withdraw(2000))    # ✅ Withdrawn ₹2000
print(acc1.add_interest())    # ✅ Interest added
```

```
print(acc1.get_balance())     # 💰 Balance: ₹13650.0

# Transfer between accounts
print(acc1.transfer(3000, acc2))  # ✅ Transferred

# Statement
acc1.get_statement()
```

⚙️ **INSTANCE METHOD vs OTHER METHODS**

python

```
class Demo:

    class_var = "I am class variable"

    def __init__(self, value):
        self.value = value      # instance variable

    # ── INSTANCE METHOD ──────────────────────
    # → Has 'self' parameter
    # → Can access instance variables
    # → Can access class variables
    # → Called on OBJECT
    def instance_method(self):
        print(f"Instance var: {self.value}")
        print(f"Class var: {self.class_var}")

    # ── CLASS METHOD ────────────────────────
    # → Has 'cls' parameter
    # → Can ONLY access class variables
    # → CANNOT access instance variables
    # → Called on CLASS or object
```

```
    @classmethod
    def class_method(cls):
        print(f"Class var: {cls.class_var}")
        # print(cls.value)  # ❌ Error!

    # ── STATIC METHOD ──────────────────────────
    # → No self or cls parameter
    # → Cannot access instance OR class vars
    # → Independent utility function
    # → Called on CLASS or object
    @staticmethod
    def static_method():
        print("I am independent!")
        # print(self.value)     # ❌ Error!
        # print(cls.class_var)  # ❌ Error!


obj = Demo("Hello")

# Calling methods
obj.instance_method()     # called on OBJECT
Demo.class_method()       # called on CLASS
Demo.static_method()      # called on CLASS
obj.static_method()       # also works on object
```

🔄 **INSTANCE METHOD TYPES**

1. Getter Method:

python

```
class Person:
    def __init__(self, name, age):
        self.__name = name    # private
```

```
        self.__age  = age     # private

    # GETTER → returns private variable value
    def get_name(self):
        return self.__name

    def get_age(self):
        return self.__age



p = Person("Rahul", 25)
print(p.get_name())   # Rahul
print(p.get_age())    # 25
```

2. Setter Method:

python

```
class Person:
    def __init__(self, name, age):
        self.__name = name
        self.__age  = age

    # SETTER → validates and sets private variable
    def set_name(self, name):
        if len(name) > 2:
            self.__name = name
        else:
            print("❌ Name too short!")

    def set_age(self, age):
        if 0 < age < 120:
            self.__age = age
        else:
```

```
        print("❌ Invalid age!")


p = Person("Rahul", 25)
p.set_name("Priya")   # ✅ updated
p.set_name("X")       # ❌ Name too short!
p.set_age(200)        # ❌ Invalid age!
```

3. Action Method:

python

```
class Robot:
    def __init__(self, name):
        self.name    = name
        self.battery = 100
        self.tasks   = []

    # ACTION methods → perform operations
    def walk(self):
        self.battery -= 10
        print(f"{self.name} is walking 🚶 ")

    def talk(self, message):
        self.battery -= 5
        print(f"{self.name} says: {message} 🤖 ")

    def charge(self):
        self.battery = 100
        print(f"{self.name} fully charged ⚡ ")

    def status(self):
        print(f"Battery: {self.battery}%")
```

```
r = Robot("R2D2")
r.walk()        # R2D2 is walking 🚶
r.talk("Hello!")  # R2D2 says: Hello! 🤖
r.status()       # Battery: 85%
r.charge()       # fully charged ⚡
```

🧪 **SELENIUM REAL EXAMPLE**

```python
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class LoginPage:

    # Class variable (shared locators)
    PAGE_URL = "https://app.example.com/login"

    def __init__(self, driver):
        # Instance variables
        self.driver   = driver
        self.wait     = WebDriverWait(driver, 10)
        self._timeout = 10

        # Locators as instance variables
        self.username_field = (By.ID, "username")
        self.password_field = (By.ID, "password")
        self.login_button   = (By.ID, "loginBtn")
        self.error_message  = (By.ID, "errorMsg")
```

```
# —— INSTANCE METHODS ————————————————

def open_page(self):
    """Navigate to login page"""
    self.driver.get(self.PAGE_URL)

def enter_username(self, username):
    """Type username"""
    field = self.wait.until(
        EC.visibility_of_element_located(
            self.username_field
        )
    )
    field.clear()
    field.send_keys(username)

def enter_password(self, password):
    """Type password"""
    field = self.driver.find_element(
        *self.password_field
    )
    field.clear()
    field.send_keys(password)

def click_login(self):
    """Click login button"""
    self.driver.find_element(
        *self.login_button
    ).click()
```

```
    def login(self, username, password):
        """Complete login flow"""
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()

    def get_error_message(self):
        """Get error message text"""
        return self.driver.find_element(
            *self.error_message
        ).text

    def is_login_successful(self):
        """Check if login was successful"""
        return "dashboard" in self.driver.current_url


# ── IN TEST ──────────────────────────────────
driver = webdriver.Chrome()
login  = LoginPage(driver)      # instantiation

# Using instance methods
login.open_page()
login.login("admin@test.com", "password123")

if login.is_login_successful():
    print("✅ Login successful!")
else:
    print(f"❌ Error: {login.get_error_message()}")
```

📊 **COMPLETE SUMMARY TABLE**

Feature	Instance Variable	Instance Method
Definition	Data stored in object	Function in object
First param	—	self
Access	self.var or obj.var	obj.method()
Unique per object	✅ Yes	✅ Yes
Can access instance vars	—	✅ Yes
Can access class vars	✅ Yes	✅ Yes
Created when	__init__ runs	Class is defined
Modified by	Direct or setter	Action methods

💬 **INTERVIEW ANSWER**

Q: What are instance variables and
instance methods in Python OOP?

A: Instance Variables:
→ Variables defined inside __init__
using self keyword
→ Each object gets its OWN separate copy
→ Values differ from object to object
→ Represent the STATE of an object
Example: self.name, self.age, self.balance

Instance Methods:
→ Functions defined inside a class
that take 'self' as first parameter
→ Can access and modify instance variables
→ Called on an object (not the class)
→ Represent the BEHAVIOR of an object
Example: deposit(), withdraw(), login()

Key Point:

→ Instance variables = WHAT an object HAS

→ Instance methods = WHAT an object DOES

💡 **One line to remember: "Instance Variables = Object's DATA 📦 , Instance Methods = Object's ACTIONS ⚙️ "**